
Einführung in Python

Marc Hoffmann
TU Darmstadt

Modellierungswoche 2026

04. Oktober

Was lernen wir heute?

Nach dem Kurs könnt ihr einfache Python-Programme schreiben.

Dafür werden wir:

- Variablen verwenden und mit ihnen rechnen,
- mit Listen arbeiten,
- Entscheidungen mit `if` treffen,
- Wiederholungen mit `for` und `while` schreiben,
- eigene Funktionen definieren,
- einfache Bibliotheken importieren,
- NumPy und seine Vorzüge kennenlernen.

Variablen

Variablen speichern Werte.

```
alter = 21  
name = "Anna"  
temperatur = 18.5
```

Wichtig:

- keine Leerzeichen in Variablennamen
- Groß- und Kleinschreibung wird unterschieden

```
alter = 21  
Alter = 30
```

Das sind zwei verschiedene Variablen.

Zuweisung ist keine Gleichheit

```
a = 3  
b = a  
a = 5
```

Danach gilt:

$a = 5, \quad b = 3$

= bedeutet in Python:

Speichere den Wert rechts in der Variable links.

Datentypen

Einige wichtige Datentypen:

6	# int
3.14	# float
"Hallo"	# str
True	# bool
False	# bool

print und f-Strings

Wert der Variable x und deren Typ anzeigen:

```
x = 6
print(x)
print(type(x))
```

Variablen direkt in Text einsetzen:

```
name = "Anna"
alter = 21

print(f"Hallo {name}!")
print(f"Du bist {alter} Jahre alt.")
```

Auch Rechnungen sind möglich:

```
print(f"Kommendes Jahr bist du {alter + 1}.")
```

Rechnen mit Python

Python kann direkt als Taschenrechner verwendet werden:

```
3 + 4
7 - 2
5 * 6
8 / 2
```

Weitere Operatoren:

```
7 // 2      # ganzzahlige Division
7 % 2       # Rest
2 ** 3      # Potenz
```

Auch Dezimalzahlen (Datentyp float) erlaubt!

Operatoren im Überblick

Operator	Bedeutung
+	Addition
-	Subtraktion
*	Multiplikation
/	Division
//	ganzzahlige Division
%	Rest
**	Potenz

17 // 5 # 3

17 % 5 # 2

Reihenfolge der Berechnung

Ganz normales Punkt vor Strich:

$$2 + 3 * 4$$

ergibt

14

Klammern ändern richtigerweise die Reihenfolge:

$$(2 + 3) * 4$$

ergibt

20

Variablen und Rechenoperationen

```
preis = 12  
anzahl = 3
```

```
gesamt = preis * anzahl
```

```
print(gesamt)
```

Variablen können in Berechnungen verwendet werden.

Miniübung: Taschenrechner

Definiert zwei Variablen und weist einer den Wert 17 und der anderen den Wert 5 zu.

Gibt aus:

- Summe
- Differenz
- Produkt
- Quotient
- ganzzahliger Anteil
- Rest

Vergleiche

Python kann Werte vergleichen:

```
x == 10
```

```
x != 10
```

```
x > 10
```

```
x < 10
```

```
x >= 10
```

```
x <= 10
```

Das Ergebnis ist:

True

oder

False

if

Mit `if` können wir Entscheidungen treffen:

```
x = 7
```

```
if x > 0:  
    print("positiv")
```

Wichtig:

- Doppelpunkt :
- Einrückung
- eingerückter Code gehört zum `if`

if und else

```
x = -3
```

```
if x > 0:  
    print("positiv")  
else:  
    print("nicht positiv")
```

Mehrere Fälle mit elif

```
x = 0

if x > 0:
    print("positiv")
elif x < 0:
    print("negativ")
else:
    print("null")
```

Logische Operatoren

Bedingungen können kombiniert werden.

`and`
`or`
`not`

Beispiele:

```
alter >= 18 and besitzt_fuehrerschein  
x < 0 or x > 100
```

Miniübungen

Miniübung 1: Gerade oder ungerade?

Definiere eine Zahl und gib aus, ob sie gerade oder ungerade ist.

Miniübung 2: Zahl klassifizieren

Definiere eine Zahl.

Gib aus:

- positiv
- negativ
- oder null

Extra: Unterscheide zusätzlich zwischen positiv/gerade und positiv/ungerade.

Listen

Eine Liste enthält mehrere Werte:

```
zahlen = [10, 20, 30, 40]
```

Auch verschiedene Typen sind möglich:

```
liste = [42, "Hallo", 3.14, True]
```

Zugriff auf Listenelemente

Die Zählung beginnt bei **0**.

```
zahlen = [10, 20, 30, 40]
```

```
zahlen[0]      # 10
```

```
zahlen[1]      # 20
```

```
zahlen[3]      # 40
```

Das letzte Element:

```
zahlen[-1]
```

Slicing

Teile einer Liste auswählen:

```
zahlen = [10, 20, 30, 40, 50]
```

```
zahlen[1:4]
```

ergibt:

```
[20, 30, 40]
```

Außerdem:

```
zahlen[:3]
```

```
zahlen[2:]
```

Listen verändern

Element ändern:

```
zahlen[0] = 99
```

Element hinzufügen:

```
zahlen.append(60)
```

Element entfernen:

```
zahlen.remove(20)
```

Nützliche Listenoperationen

Länge der Liste:

```
len(zahlen)
```

Ist 50 enthalten?

```
50 in zahlen # Datentyp bool
```

Element (hinten) hinzufügen:

```
zahlen.append(60)
```

Eine wichtige Überraschung

Was passiert hier?

```
a = [1, 2, 3]  
b = [4, 5, 6]
```

```
print(a + b)
```

Ergebnis:

```
[1, 2, 3, 4, 5, 6]
```

Nicht:

```
[5, 7, 9]
```

Bei Python-Listen bedeutet +:

Listen zusammenhängen

Miniübung: Einkaufsliste

5 Minuten

```
einkauf = ["Apfelsaft", "Brot", "Erdbeeren"]
```

Aufgaben:

- 1 erstes Element ausgeben
- 2 letztes Element ausgeben
- 3 "Käse" hinzufügen
- 4 "Brot" entfernen
- 5 Länge ausgeben

Warum Schleifen?

Wir möchten oft denselben Code mehrfach ausführen.

Statt:

```
print(1)
print(2)
print(3)
print(4)
```

können wir schreiben:

```
for x in [1, 2, 3, 4]:
    print(x)
```

for über eine Liste

```
fruechte = ["Apfel", "Birne", "Banane"]
```

```
for frucht in fruechte:  
    print(frucht)
```

frucht nimmt nacheinander die Werte der Liste an.

Akkumulatoren

Wir können mit einer Schleife Werte aufsummieren:

```
zahlen = [2, 3, 5, 7]
```

```
summe = 0
```

```
for x in zahlen:  
    summe += x # entspricht "summe = summe + x"
```

```
print(summe)
```

Schleife + Bedingung

```
zahlen = [3, 8, 2, 11, 14, 7]
```

```
for x in zahlen:  
    if x % 2 == 0:  
        print(x)
```

Hier kombinieren wir:

- Liste
- for
- if
- %

range()

`range(5)`

liefert die Werte

0, 1, 2, 3, 4

Mit Start und Ende:

`range(2, 7)`

liefert

2, 3, 4, 5, 6

Mit Schrittweite:

`range(2, 10, 2)`

liefert:

2, 4, 6, 8

range() in Action

Beispiel:

```
fruechte = ["Apfel", "Birne", "Banane"]
for i in range(len(fruechte)):
    if fruechte[i] == "Birne" :
        fruechte[i] = "Ananas"
print(fruechte)
```

Was wird ausgegeben?

Miniübung: Summe und Mittelwert

5 Minuten

```
zahlen = [4, 7, 2, 9, 5, 14]
```

Berechne mit einer Schleife:

- 1 die Summe
- 2 den Mittelwert

Hinweis:

```
len(zahlen)
```

Extra: Berechne mit einer Schleife Summe und Mittelwert der geraden Zahlen in der liste.

enumerate()

Wenn wir Index **und** Wert brauchen:

```
fruechte = ["Apfel", "Birne", "Banane"]
```

```
for i, frucht in enumerate(fruechte):  
    print(i, frucht)
```

Ausgabe:

```
0 Apfel  
1 Birne  
2 Banane
```

break und continue

break beendet eine Schleife:

```
for x in zahlen:  
    if x > 10:  
        break
```

continue überspringt einen Durchlauf:

```
for x in zahlen:  
    if x < 0:  
        continue  
    print(x)
```

while-Schleifen

for: Wiederhole für jedes Element / jede Zahl.

while: Wiederhole, solange eine Bedingung wahr ist.

Beispiel:

```
x = 0
while x < 5:
    print(x)
    x += 1
```

Was wird ausgegeben?

Warum Funktionen?

Statt Code mehrfach zu schreiben:

```
print(3 * 4)
print(8 * 2)
print(5 * 7)
```

können wir eine Funktion definieren:

```
def rechteck_flaeche(a, b):
    return a * b
```

Aufruf:

```
print(rechteck_flaeche(3, 4))
```

Funktionen

Funktionen müssen definiert werden bevor (oberhalb) sie aufgerufen werden.

```
def quadrat(x):  
    return x ** 2
```

Aufruf:

```
y = quadrat(5)  
print(y)
```

x ist ein **Parameter**.

5 ist das beim Aufruf übergebene **Argument**.

return gibt ein Ergebnis zurück.

Miniübung: Eigene Funktionen

Schreibt:

```
def ist_gerade(n):  
    ...
```

Die Funktion soll True oder False zurückgeben.

```
print(ist_gerade(8))  
print(ist_gerade(5))
```

Extra:

```
def maximum(a, b):  
    ...
```

Python kann erweitert werden

Python besitzt viele Bibliotheken.

Eine Bibliothek wird typischerweise ganz oben im code importiert:

```
import numpy
```

Danach:

```
numpy.sqrt(25)
```

import macht zusätzliche Funktionen verfügbar.

NumPy importieren

Für numerische Berechnungen verwenden wir häufig NumPy:

```
import numpy as np
```

np ist einfach ein kurzer Name für numpy.

Jetzt:

```
a = np.array([1, 2, 3])
```

Liste oder NumPy-Array?

Python-Liste:

```
a = [1, 2, 3]
b = [4, 5, 6]
```

```
a + b
```

liefert:

```
[1, 2, 3, 4, 5, 6]
```

NumPy:

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
```

```
a + b
```

liefert:

```
[5 7 9]
```

Elementweise Rechnungen

```
a = np.array([1, 2, 3])
```

```
2 * a
```

ergibt:

```
[2 4 6]
```

Auch:

```
a ** 2
```

ergibt:

```
[1 4 9]
```

Die Operation wird auf die einzelnen Elemente angewendet.

NumPy in action

```
import numpy as np

zahlen = [1, 2, 3]

a = 2*np.array(zahlen)

for x in a:
    if x % 2 == 1:
        print(x ** 2)
```

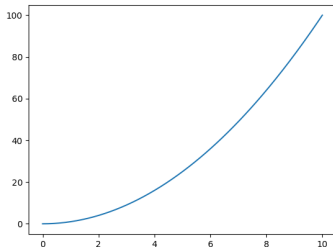
Was wird ausgegeben?

matplotlib

```
import numpy as np
import matplotlib.pyplot as plt

def quadrat(x) :
    return x**2
xi = np.linspace(0,10,200)
plt.plot(xi,quadrat(xi))
plt.show()
```

Ausgabe:



Abschlussaufgabe: Primzahlen

Schreibe eine Funktion, die alle Primzahlen zwischen `start` und `end` finden und rufe sie für verschiedene Werte von `start` und `end` auf.

Dafür brauchen wir:

- `for`
- `range`
- `%`
- `if`
- `break`
- Listen

Diese Aufgabe verbindet fast alles, was wir heute gelernt haben.